

Trustworthy Machine Learning – Spring 2026

Homework Assignment 1

ID: 205455207

Question #1 – White-box vs. query-based black-box attack

1. Benign accuracy. After implementing `utils.compute_accuracy`, the (benign) accuracy of `simple-cnn-0` on the supplied 200-image test set is

$$\boxed{\text{acc} = 0.8750}.$$

2. White-box PGD success rates. Running `main_a.py` with our `PGDAttack` ($\epsilon = 8/255$, $\alpha = 1/255$, 50 iterations, random initialisation, early stopping) yields the following success rates against `simple-cnn-0`:

Attack	Success rate
Untargeted PGD (white-box)	0.9850
Targeted PGD (white-box)	0.9400

For targeted attacks, target labels are drawn at random with $t = (c_x + \text{randint}(1, n_{\text{classes}})) \bmod n_{\text{classes}}$, guaranteeing $t \neq c_x$.

3. Black-box NES-PGD success rates. We implemented `NESBoxPGDAttack` using anti-thetic sampling: at every PGD iteration we draw $k = 200$ Gaussian noise tensors u_i and query the model on $x \pm \sigma u_i$ (so a total of $2k = 400$ queries per iteration per active sample). The gradient is approximated as

$$\widehat{\nabla_x \text{loss}} = \frac{1}{2k\sigma} \sum_{i=1}^k [\text{loss}(x + \sigma u_i) - \text{loss}(x - \sigma u_i)] u_i.$$

With momentum μ the update used at iteration t is $g_t = \mu g_{t-1} + \widehat{\nabla_x \text{loss}}_t$, and PGD takes the sign of g_t . Results (success rates and median number of model queries):

	Untargeted		Targeted	
	success	median #queries	success	median #queries
NES-PGD, $\mu = 0.0$	0.9250	3600	0.7500	7600
NES-PGD, $\mu = 0.9$	0.9650	2400	0.8650	4000

Comparison with white-box. The black-box NES attack is markedly less query-efficient than white-box PGD, which only needs one back-prop per iteration; the black-box attack also tends to have slightly lower success rates (especially for targeted attacks), since the NES gradient estimate is noisier than the true gradient. In our run the white-box attack reaches 0.9850/0.9400 (untargeted/targeted) while the best black-box configuration reaches 0.9650/0.8650.

Effect of momentum. Adding momentum $\mu = 0.9$ to the NES-PGD attack *both raises the success rate and reduces the number of model queries*: the success rate climbs from 0.9250 to 0.9650 (untargeted, +4.0 p.p.) and from 0.7500 to 0.8650 (targeted, +11.5 p.p.), while the median number of queries drops from 3600 to 2400 (untargeted) and from 7600 to 4000 (targeted). Intuitively, the NES gradient estimator is a stochastic finite-difference estimator – each iteration produces a noisy estimate of the same underlying gradient. Accumulating a fraction of the previous gradient acts as an exponential moving average that suppresses estimation noise and reinforces directions that have been consistently useful across iterations, leading PGD to escape the eps-ball boundary along a more informative direction in fewer iterations (each iteration costs the same $2k$ queries, so fewer iterations means fewer queries).

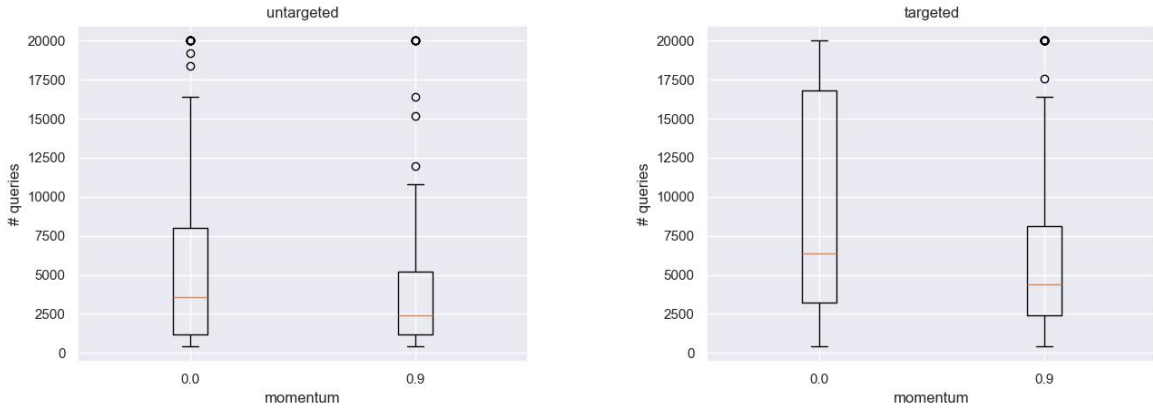


Figure 1: Number of queries used by the NES-PGD black-box attack on `simple-cnn-0`, with and without momentum, for untargeted (left) and targeted (right) attacks. Distributions over the 200 images of the test set.

Question #2 – Transferability-based black-box attack

Single-model transferability. Using our white-box PGDAttack against each of the three pre-trained CNNs, we obtain the following matrices of success rates. The (i, j) entry is the rate at which adversarial examples crafted on the surrogate `simple-cnn- $i$` (rows) succeed against the target `simple-cnn- $j$` (columns). The diagonal corresponds to white-box attacks; the off-diagonal entries quantify transferability.

Untargeted:

$$\begin{pmatrix} 0.985 & 0.570 & 0.535 \\ 0.690 & 0.965 & 0.585 \\ 0.585 & 0.545 & 0.950 \end{pmatrix}$$

Targeted:

$$\begin{pmatrix} 0.945 & 0.315 & 0.265 \\ 0.410 & 0.865 & 0.285 \\ 0.360 & 0.275 & 0.845 \end{pmatrix}$$

Discussion. Untargeted PGD adversarial examples transfer reasonably well between these three CNNs: off-diagonal success rates are in the range $[0.535, 0.690]$, well above the $1 - \text{acc} \approx 0.13$ baseline of merely returning misclassified images. In contrast, targeted attacks transfer significantly worse: the off-diagonal entries are in the range $[0.265, 0.410]$, as expected since targeting a specific (wrong) class requires the surrogate’s gradient to align with the target model’s gradient on a much narrower direction.

Ensemble attack. After implementing `PGDEnsembleAttack` (which descends the average cross-entropy loss across the ensemble at every PGD step) we craft adversarial examples against the ensemble `{simple-cnn-1, simple-cnn-2}` and evaluate them on `simple-cnn-0`:

Attack source	Untargeted SR on cnn-0	Targeted SR on cnn-0
single surrogate cnn-1	0.690	0.410
single surrogate cnn-2	0.585	0.360
ensemble {cnn-1, cnn-2} (Liu et al.)	0.7400	0.5050

Both untargeted and targeted transferability *improve* when attacking the ensemble: the untargeted success rate on `simple-cnn-0` climbs from $\max(0.690, 0.585) = 0.690$ to 0.7400 (an absolute boost of +5.0 percentage points), and the targeted success rate climbs from 0.410 to 0.5050 (+9.5 p.p.). Intuitively, an adversarial example crafted to fool *multiple* surrogates simultaneously must exploit features that are common to several models, and is therefore more likely to exploit features shared with an unseen target as well; this shared, less model-specific perturbation generalises better to the held-out model `simple-cnn-0`, exactly as predicted by Liu et al. [3].

Question #3 – Bit-flip attacks

We implement IEEE-754 single-precision (`float32`) bit conversion helpers (`utils.binary` and `utils.float32`) and a random bit-flip routine (`utils.random_bit_flip`). We then iterate over the five layers of `simple-cnn-1` (`conv1`, `conv2`, `fc1`, `fc2`, `fc3`) and, for each layer, perform `BF_PER_LAYER = 450` bit flips: at each step we sample a random weight and a random bit index in $\{0, \dots, 31\}$, flip the bit, measure the relative accuracy drop

$$\text{RAD} = \frac{\text{acc}_o - \text{acc}_{bf}}{\text{acc}_o},$$

and restore the weight. The original (pre-flip) accuracy of `simple-cnn-1` on the test set is $\text{acc}_o = 0.8250$. In total $450 \times 5 = 2250$ bit-flips were performed.

1. **Maximum RAD:** 0.7273.
2. **Fraction of bits with RAD > 15%:** 2.62% of all 2250 flips.
3. **Per-bit median RAD.** The box-plot in Figure 2 summarises the RAD distribution as a function of the flipped bit index (big-endian: index 0 is the sign bit, indices 1–8 are the exponent, indices 9–31 are the mantissa). The bit with the highest median RAD is index 1, corresponding to the **most-significant bit of the IEEE-754 exponent**.

Why is the MSB of the exponent the most damaging? The IEEE-754 binary32 format encodes a value as $x = (-1)^s (1 + m) \cdot 2^{e-127}$, where s is the sign bit, $e \in \{0, \dots, 255\}$ is the (biased) exponent, and $m \in [0, 1)$ is the mantissa. Trained neural-network weights are typically of magnitude $\ll 1$, so their unbiased exponent is negative and the biased exponent e is comfortably below 127, meaning the MSB of e (i.e. the $2^7 = 128$ bit of the exponent field) is almost always 0. Flipping this bit from 0 to 1 *adds 128 to the exponent* and multiplies the weight by $2^{128} \approx 3.4 \times 10^{38}$ – typically the value becomes `inf` or saturates downstream activations to `NaN/Inf`, devastating the model’s prediction. By contrast, flipping a mantissa bit changes the weight by at most a factor of ~ 2 (and usually far less) and flipping the sign bit just negates a small number; both have a much milder effect on accuracy.

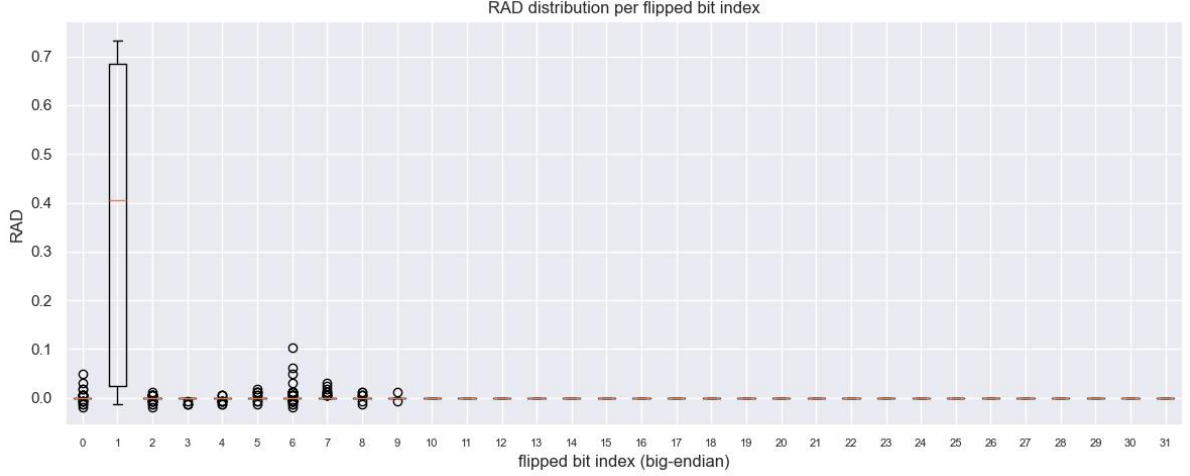


Figure 2: Box-plot of RAD distribution per flipped bit index (big-endian IEEE-754, bit 0 = sign, bits 1–8 = exponent, bits 9–31 = mantissa) on `simple-cnn-1`. Bit 1 (the MSB of the exponent) has the highest median RAD.

References

- [1] S. Hong, P. Frigo, Y. Kaya, C. Giuffrida, and T. Dumitras. *Terminal brain damage: Exposing the graceless degradation in deep neural networks under hardware fault attacks*. Proc. USENIX Security, 2019.
- [2] A. Ilyas, L. Engstrom, A. Athalye, and J. Lin. *Black-box adversarial attacks with limited queries and information*. Proc. ICML, 2018.
- [3] Y. Liu, X. Chen, C. Liu, and D. Song. *Delving into transferable adversarial examples and black-box attacks*. Proc. ICLR, 2017.